

UNIVERSIDADE FEDERAL DE ITAJUBÁ

Engenharia de Dados na Prática

Limpeza avançada e engenharia de atributos

Profa. Bárbara Pimenta Caetano

O que veremos hoje

- 1 Uma terceira fonte, para termos tempo no projeto
- 2 Tipos: quando a coluna passa a se comportar
- 3 Texto: o mesmo valor escrito de várias formas
- 4 Datas: o que o computador entende por tempo
- 5 Atributos derivados e o critério para criá-los

Onde estamos



Bronze

Três fontes ao fim de hoje



Prata

Correta na aula passada, utilizável hoje

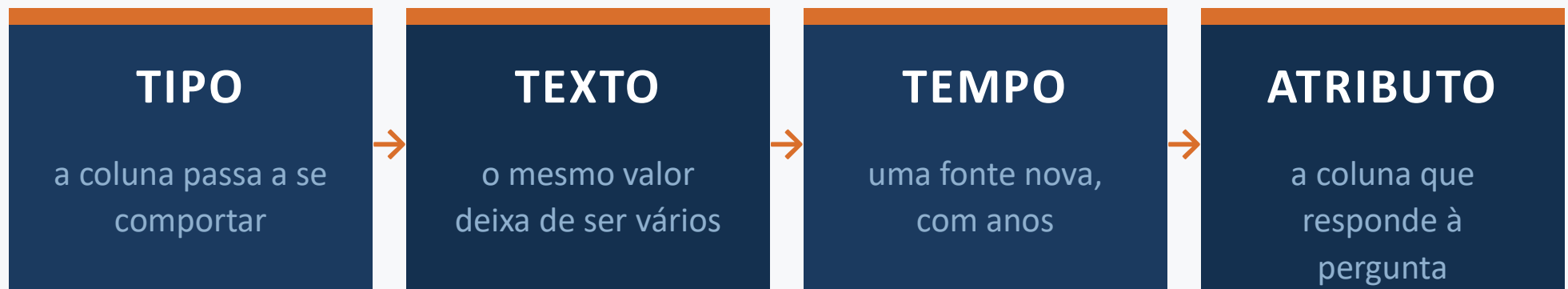


Ouro

Modelagem e entrega

Correto e utilizável não são a mesma coisa.

Quatro coisas para a Prata ficar utilizável



Nenhuma delas corrige erro. A Prata já está correta desde a aula passada. Hoje ela deixa de ser apenas correta.

No fim da aula, as funções genéricas saem dos scripts e passam a morar em um módulo só.

O que a sua Prata ainda não é

Os tipos estão frouxos

Categoria guardada como texto livre, número guardado como texto, nada impede um valor impossível de entrar.

O mesmo valor aparece escrito de formas diferentes

E o computador trata cada forma como uma coisa distinta na hora de agrupar ou de juntar com outra fonte.

Não existe nenhuma coluna que responda à sua pergunta

Só existem as colunas que a fonte resolveu publicar. Ninguém lá sabia o que você queria perguntar.

As três coisas se resolvem hoje, nessa ordem.

A sua Prata, hoje

Quatro linhas do arquivo que você gravou na aula passada:

id	name	region.value	incomeLevel.value	longitude
BRA	Brazil	Latin America & Caribbean	Upper middle income	-47.9292
CHN	China	East Asia & Pacific	Upper middle income	116.286
IND	India	South Asia	Lower middle income	77.225
NOR	Norway	Europe & Central Asia	High income	10.7387

Está correta	Sem duplicata, sem agregado, sem espaço sobrando.
E ainda assim é texto	Para o computador, a coluna destacada é texto livre. Nada impede 'Hgi income'.
E não responde nada	Nenhuma dessas colunas foi criada para responder à sua pergunta.

Linhas reais da API do Banco Mundial.

O que o pandas adivinhou, e o que deveria ser

coluna	tipo agora	o que deveria ser
id	object	texto, é identificador
name	object	texto
region.value	object	categoria
incomeLevel.value	object	categoria COM ordem
longitude	float64	float64, já está certo

object é o tipo que o pandas usa quando desiste de adivinhar. Ele aceita qualquer coisa, e é exatamente por isso que não protege nada.

Três das cinco colunas estão guardadas de um jeito que não corresponde ao que elas são.

O que muda quando o tipo muda

As mesmas quatro longitudes, ordenadas de duas formas:

Ordenado como texto

-47.9292

10.7387

116.286

77.225

Ordenado como número

-47.9292

10.7387

77.225

116.286

Como texto, 116 vem antes de 77, porque o 1 vem antes do 7. A comparação é feita caractere a caractere, e nenhum erro é acusado.

Longitudes reais de Brasil, Noruega, Índia e China.

Categoria com ordem, e os dois intrusos



A escala tem ordem natural, e declarar isso faz comparar e ordenar funcionarem.

A mesma coluna traz também 'Aggregates' e 'Not classified'. Não são níveis de renda. Ao declarar a escala, os dois viram ausentes, e é isso que você quer: o código passa a dizer em voz alta o que estava escondido no meio do texto.

Os quatro níveis e os dois intrusos aparecem na resposta real da API.

O mesmo valor, escrito de várias formas

Real, na nossa fonte, antes da limpeza da aula passada:

valor na fonte	o que parece	o que é
'Latin America & Caribbean '	igual ao de baixo	tem espaço no fim
'Latin America & Caribbean'	igual ao de cima	não tem

Dentro de um arquivo só, isso quase não incomoda. O custo aparece na junção: a outra fonte escreve sem o espaço, e nenhuma linha casa.

Ilustrativo, para o caso de a sua fonte ser em português:

São Paulo

SAO PAULO

sao paulo

S. Paulo

SP

Uma cidade para você. Cinco cidades para o computador.

Uma coluna para comparar, outra para mostrar

rótulo original	chave_texto	serve para
'São Paulo'	sao paulo	juntar e agrupar
'SAO PAULO'	sao paulo	juntar e agrupar
' São Paulo '	sao paulo	juntar e agrupar

```
def chave_texto(serie):  
    s = serie.str.strip().str.lower()  
    s = s.str.normalize("NFKD")  
    s = s.str.encode("ascii", errors="ignore")  
    return s.str.decode("utf-8")
```

Repare no nome: `chave_texto`, não `limpar_texto`. O rótulo original fica na tabela. Quem some com o acento na hora de exibir está jogando fora informação.

Falta tempo nos nossos dados

Nenhuma das duas fontes tem coluna temporal

A tabela de países descreve um retrato, não uma história. A de artistas traz um ano de estreia, que é um atributo, não uma linha do tempo.

Sem tempo, você não pergunta:

- cresceu ou caiu?
- quando mudou de patamar?
- o que aconteceu antes e o que veio depois?

Boa parte das perguntas interessantes é sobre variação. Vamos buscar uma fonte que tenha tempo.

A mesma API, outro recurso

Indicador por país e por ano. Aqui, população total:

```
https://api.worldbank.org/v2/country/all  
/indicator/SP.POP.TOTL  
?format=json&date=2000:2023
```

Um registro da resposta:

```
{"indicator": {"id": "SP.POP.TOTL", ...},  
  "country": {"id": "ZH", "value": "Africa Eastern..."},  
  "countryiso3code": "AFE",  
  "date": "2023",  
  "value": 750491370,  
  "unit": "", "obs_status": "", "decimal": 0}
```

Resposta real da API, consultada durante o preparo desta aula.

O alerta de paginação disparou



você levou esta e deixou estas para trás

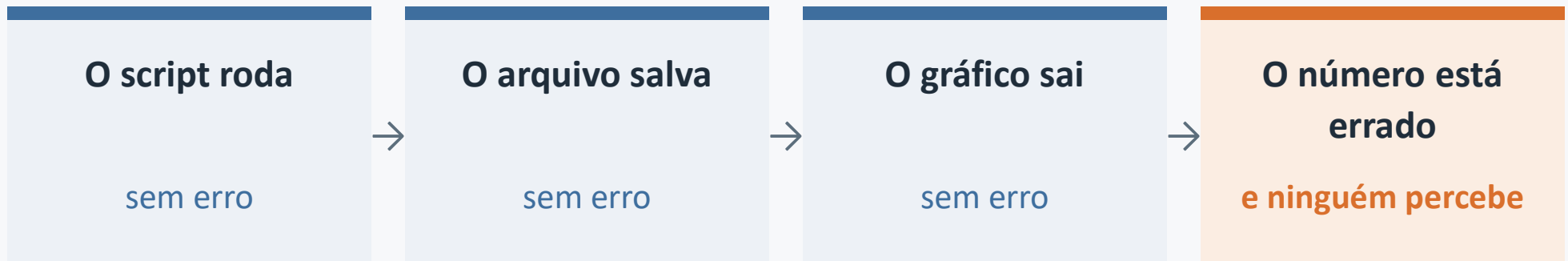
Na aula de ingestão vocês escreveram um alerta que nunca tinha disparado:

```
if meta["pages"] > 1:  
    print("ATENCAO: a resposta veio paginada.")
```

Nesta fonte ele dispara. O metadado real:

```
{"page": 1, "pages": 133, "total": 530,  
  "lastupdated": "2026-07-13"}
```

O defeito que não avisa



Ignorar a paginação não produz mensagem de erro em lugar nenhum. Você leva para casa a primeira página achando que tem a série inteira.

Toda vez que uma fonte responde em pedaços, confira o total recebido contra o total que ela informou. São duas linhas de código e evitam um trabalho inteiro construído sobre número errado.

Categoria com ordem é diferente de categoria sem ordem

A tabela de países traz o nível de renda, que tem ordem natural:

```
ordem = ["Low income", "Lower middle income",  
         "Upper middle income", "High income"]  
  
df["incomeLevel.value"] = pd.Categorical(  
    df["incomeLevel.value"], categories=ordem, ordered=True)
```

Os valores que não estão na lista viram ausentes. E é isso que você quer: 'Aggregates' e 'Not classified' não são níveis de renda, e o código passa a dizer isso em voz alta.

Com ordem declarada, comparar e ordenar passam a funcionar. Sem ela, o pandas ordena em ordem alfabética, e 'High income' vem primeiro.

O mesmo valor, escrito de seis formas

São Paulo
SAO PAULO
sao paulo
São Paulo
S. Paulo
SP

Para você

É a mesma cidade.

Para o computador

São seis cidades diferentes. Agrupar dá seis linhas, e juntar com outra fonte encontra no máximo uma.

Exemplo ilustrativo, montado para a aula. As nossas fontes vêm em inglês e razoavelmente padronizadas, mas a sua pode não vir.

Esse é o defeito que mais estraga junção de fontes, e é invisível em qualquer inspeção rápida, porque os valores parecem certos um a um.

Quatro operações, nessa ordem

1

Tirar espaços

Nas pontas e os repetidos no meio.

2

Unificar a caixa

Tudo em minúscula para comparar. O rótulo bonito se recupera depois.

3

Tratar acentos

Só quando a fonte é inconsistente. Se ela acentua sempre, remover acento é perder informação.

4

Mapear sinônimos

Um dicionário explícito, escrito por você, ligando cada variante ao valor canônico.

As três primeiras são automáticas. A quarta exige que você conheça o assunto, e por isso precisa ficar escrita no código, não na sua cabeça.

Nem todo dado temporal é uma data

o que a fonte manda	se converter para data	o que isso significa
date = "2023"	2023-01-01 00:00:00	um dia que ninguém mediu
date = "2022"	2022-01-01 00:00:00	e uma hora também

Um ano não tem mês, não tem dia, não tem hora e não tem fuso. Converter para data não acrescenta informação: acrescenta precisão falsa.

```
df["date"] = pd.to_numeric(  
    df["date"], errors="coerce").astype("Int64")
```

Int64 com I maiúsculo é o inteiro do pandas que aceita ausente. Para série anual, inteiro é o tipo honesto: ordena, subtrai e compara sem inventar nada.

Quando é data mesmo, o risco é o formato

```
pd.to_datetime("03/04/2026")
```

```
# 3 de abril ou 4 de março?
```

```
pd.to_datetime(coluna, format="%d/%m/%Y")
```

```
# agora não há dúvida
```

Ambiguidade de ordem

Sem format declarado, a biblioteca adivinha, e pode adivinhar diferente em arquivos diferentes da mesma fonte.

Fuso horário

Só importa quando o dado tem hora e vem de sistemas em lugares diferentes. Registro de evento e log quase sempre têm.

A data de verdade estava no metadado

"lastupdated": "2026-07-13"

Não é coluna do dado. É a data em que a fonte atualizou a série pela última vez.

Lembra da quarta pergunta?

Atualidade: o dado ainda vale?

Na aula passada era conceito. Agora existe o número que responde.

Onde ele mora

Na proveniência, não na tabela.

É informação sobre o conjunto inteiro. Repetir em toda linha seria desperdício e confusão.

Quase todo mundo joga metadado fora. Ele costuma valer tanto quanto o dado.

Atributo derivado

Uma coluna nova, calculada a partir das que já existem, que responde melhor à sua pergunta do que qualquer uma delas sozinha.

A fonte publicou o que ela achou útil. Ninguém lá sabia o que você queria perguntar.

população em 2000 e população em 2023

→ **crescimento percentual no período**

data de nascimento

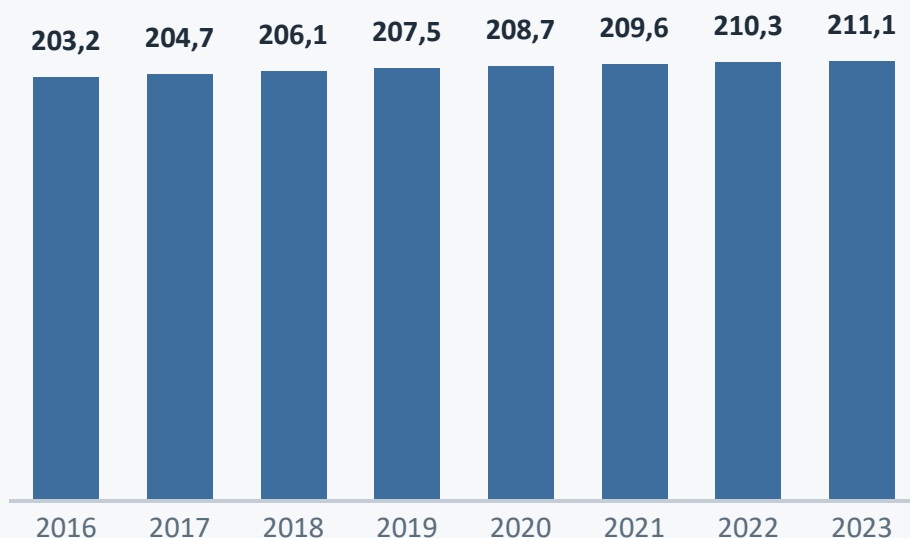
→ **faixa etária**

valor total e quantidade

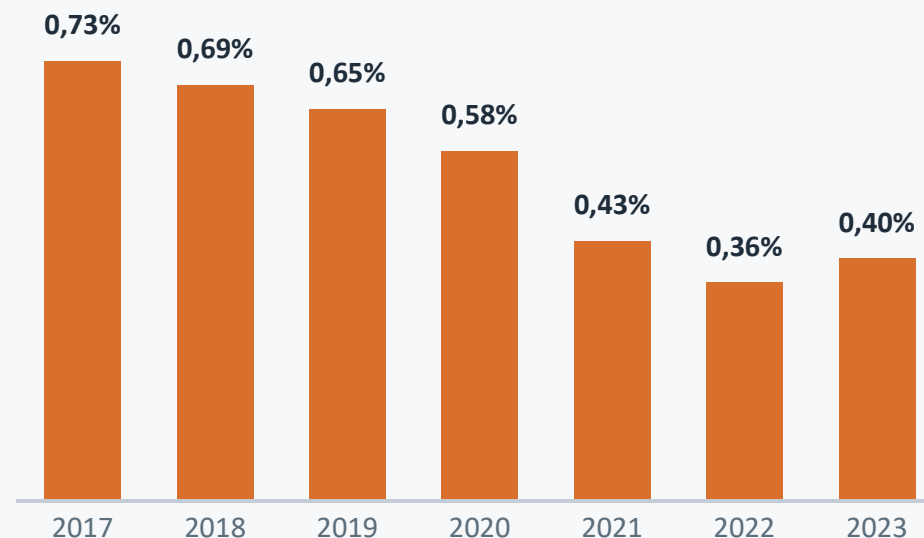
→ **valor médio por unidade**

A coluna original esconde o que a derivada mostra

População do Brasil, em milhões



Variação em relação ao ano anterior



À esquerda, a população sobe todo ano e as barras parecem iguais. À direita, a mesma série mostra o crescimento caindo pela metade em sete anos. É o mesmo dado.

Série real do Banco Mundial, indicador SP.POP.TOTL, Brasil, 2016 a 2023. A variação do primeiro ano não existe, por definição.

O critério, e a armadilha

Se você não consegue dizer em uma frase por que a coluna nova ajuda a responder a sua pergunta, ela não deveria existir.

coluna derivada	veredito	por quê
<code>proporcao = Lead / Total</code>	vale	compara, então informa
<code>soma = Lead + Feature</code>	não vale	é o Total com outro nome

Vocês descobriram no perfilamento que Total é igual a Lead mais Feature. Derivar de colunas redundantes gera uma coluna que só ocupa espaço, e que um dia vai discordar da original.

Antes de criar qualquer coluna, releia a pergunta norteadora que você escreveu na primeira entrega.

Quatro famílias para começar

Razão ou taxa

Divide para tirar o efeito do tamanho

densidade, valor por habitante, percentual do total

Variação no tempo

Compara o registro com ele mesmo no passado

crescimento anual, diferença em relação ao primeiro ano

Faixa a partir de contínuo

Agrupar para comparar em blocos

porte, faixa etária, quartil de população

Combinação de fontes

Só existe depois da junção

indicador por país cruzado com o nível de renda

O que acontece com a pasta src

antes

```
src/  
  ingerir.py  
  ingerir_paises.py  
  explorar.py  
  transformar_paises.py  
  transformar_artistas.py
```

as funcoes de texto estao
copiadas nos dois



depois

```
src/  
  ingerir.py  
  ingerir_paises.py  
  ingerir_indicador_paises.py  
  explorar.py  
  limpeza.py  
  transformar_paises.py  
  transformar_artistas.py  
  transformar_indicador.py
```

O teste para saber se algo vira módulo: a função menciona o nome de alguma fonte? Se menciona, fica no script daquela fonte. Se não menciona, é do módulo.

Por que src/limpeza.py

As funções de hoje servem para qualquer fonte. Tirar espaço, unificar caixa e mapear sinônimo não têm nada de Banco Mundial nem de Spotify.

Copiadas em cada script

Um defeito descoberto obriga a corrigir em três arquivos. Alguém esquece um, e as fontes passam a discordar.

Num módulo importado

Uma correção conserta as três de uma vez, e fica registrada em um commit só.

O teste para saber se algo vira módulo: a função menciona o nome de alguma fonte? Se menciona, fica no script daquela fonte. Se não menciona, é do módulo.

PASSO 1

Ingerir a terceira fonte

O script está pronto no material de apoio. Coloque em src e rode:

```
python src/ingerir_indicador_paises.py
```

Ele faz três coisas novas:

- Percorre todas as páginas, em vez de parar na primeira
- Guarda o lastupdated da fonte na proveniência
- Salva em dados/bronze/indicador, com a data no nome

Leia o laço de paginação antes de rodar. Ele é curto e você vai precisar dele de novo.

PASSO 2

Criar src/limpeza.py

```
"""Funcoes de limpeza que servem a qualquer fonte."""

import unicodedata

import pandas as pd

def tirar_espacos(df):
    df.columns = df.columns.str.strip()
    for c in df.select_dtypes(include="object"):
        df[c] = df[c].str.strip()
    return df
```

A função é a mesma da aula passada. Ela só mudou de casa

PASSO 3

Padronizar texto para comparação

```
def chave_texto(serie):  
    """Versao comparavel de um texto: sem acento,  
    sem espaco sobrando e tudo em minuscula."""  
    s = serie.str.strip().str.lower()  
    s = s.str.normalize("NFKD")  
    s = s.str.encode("ascii", errors="ignore")  
    return s.str.decode("utf-8")
```

Repare no nome: `chave_texto`, não `limpar_texto`. O resultado serve para comparar e juntar, não para exibir. O rótulo original continua na tabela.

PASSO 4

Mapear sinônimos explicitamente

```
def aplicar_mapa(serie, mapa):  
    """Troca variantes pelo valor canonico.  
    O que nao estiver no mapa fica como esta."""  
    return serie.replace(mapa)  
  
# no script da sua fonte, nao no modulo:  
MAPA_UF = {  
    "sao paulo": "SP",  
    "s. paulo": "SP",  
}
```

A função é genérica e mora no módulo. O dicionário é específico da sua fonte e mora no script dela.

PASSO 5

Importar o módulo no script da fonte

```
# no topo de transformar_paises.py
import limpeza

# e dentro da main, no lugar da funcao antiga:
df = limpeza.tirar_espacos(df)
```

Para o import funcionar, rode a partir da raiz do projeto e mantenha os dois arquivos em src. Se der ModuleNotFoundError, quase sempre é porque o comando foi executado de dentro da pasta errada.

PASSO 6

Tipar a coluna de nível de renda

```
# em transformar_paises.py
ORDEM_RENDA = [
    "Low income", "Lower middle income",
    "Upper middle income", "High income",
]

def tipar_renda(df):
    coluna = "incomeLevel.value"
    antes = df[coluna].isna().sum()
    df[coluna] = pd.Categorical(
        df[coluna], categories=ORDEM_RENDA, ordered=True)
    print("renda fora da escala:",
          df[coluna].isna().sum() - antes)
    return df
```

PASSO 7

Transformar a fonte nova

Crie `src/transformar_indicador_paises.py` e resolva três coisas:

O ano fica inteiro

`pd.to_numeric` na coluna `date`, e nada de converter para `data`.

As colunas vazias saem

`unit`, `obs_status` e `decimal` não carregam informação nenhuma. Conte antes de apagar e registre.

Os agregados continuam lá

E desta vez não há coluna de região para filtrá-los. Guarde a pergunta: ela se resolve na próxima aula.

PASSO 8

Primeiro atributo derivado: variação no tempo

```
def variacao_anual(df, chave, tempo, valor):  
    df = df.sort_values([chave, tempo])  
    df["variacao_pct"] = (  
        df.groupby(chave)[valor].pct_change() * 100)  
    return df  
  
df = variacao_anual(  
    df, "countryiso3code", "date", "value")
```

O `sort_values` não é enfeite. Sem ordenar primeiro, a variação é calculada contra a linha anterior do arquivo, que pode ser outro ano ou outro país.

PASSO 9

Segundo atributo: faixa a partir de contínuo

```
def faixa_por_quartil(df, coluna, rotulos):  
    df[coluna + "_faixa"] = pd.qcut(  
        df[coluna], q=4, labels=rotulos)  
    return df  
  
df = faixa_por_quartil(df, "value",  
    ["muito pequeno", "pequeno", "grande", "muito grande"])
```

qcut corta por quantidade de registros, então cada faixa fica com um quarto dos casos. cut corta por valor, em faixas de largura igual. São resultados diferentes, e a escolha depende da pergunta.

Atributo Derivados: escolha sua

Criar um atributo que sirva à pergunta norteadora que você escreveu na primeira entrega. Volte nela antes de decidir.

Ele precisa passar no critério:

- 1 Uma frase explica por que essa coluna ajuda a responder a pergunta
- 2 Ela não repete informação que já existe em outra coluna
- 3 O cálculo está em uma função, com nome que diz o que ela faz

PASSO 11

Registrar no README

```
## Atributos derivados
```

```
### variacao_pct
```

```
Variacao percentual do indicador em relacao ao ano  
anterior, por pais. Serve para comparar ritmo de  
crescimento entre paises de tamanhos diferentes.  
Ausente no primeiro ano de cada pais, por definicao.
```

```
### value_faixa
```

```
Quartil de populacao. Serve para agrupar paises  
comparaveis sem escolher corte arbitrario.
```

Para cada atributo: o que é, por que existe, e o que ele faz quando o dado não permite calcular.

PASSO 12

Rodar tudo e conferir

```
python src/transformar_paises.py  
python src/transformar_indicador_paises.py
```

Três conferências antes de commitar:

Nenhuma conversão comeu dado sem avisar

Os prints de antes e depois batem com o esperado.

Os atributos novos têm valores plausíveis

Uma variação de 4000% costuma ser divisão por um valor quase zero.

A proveniência ganhou a linha de hoje

Com as decisões novas listadas, não só as da aula passada.

Para aprofundar

Leitura principal

MCKINNEY, W. Python para análise de dados. 3. ed. Novatec, 2023. Capítulos de manipulação de texto e de séries temporais.

Documentação

Guia de dados categóricos e guia de séries temporais do pandas. São as duas páginas que respondem quase tudo de hoje.

Para o seu projeto

Catálogo de indicadores do Banco Mundial. Quem tem tema de economia, saúde ou educação provavelmente acha série pronta lá.